Appendix: Assessment cover

OXFORD BROOKES UNIVERSITY

Assessment cover

STUDENTS, PLEASE COPY THIS PAGE AND USE AS THE COVER PAGE FOR YOUR SUBMISSION

Module No:	COMP5045	Module title:	**Introduction to Artificial Intelligence**

Assessment title:	**Algorithmic Problem Solving and Knowledge Representation** **Project Report**

Due date and time:	8 December 2025, 13:00

Estimated total time to be spent on assignment:	*c0 hours per student, direct work on coursework, plus* *54 hours per student on independent study*

LEARNING OUTCOMES

On successful completion of this module, students will be able to achieve the module following learning outcomes (LOs):
LO#1 Demonstrate an understanding of the foundation of propositional and predicate logic.
LO#2 Use logic to model a knowledge domain and reason about it.
LO#3 Demonstrate an understanding of a range of algorithmic problem solving methods.
LO#4 Analyse and synthesise solutions to given real world and business problems related to algorithmic reasoning and problem solving using various techniques.

Engineering Council AHEP4 LOs assessed (from S2 2022-23)	
B1	Apply knowledge of mathematics, statistics, natural science and engineering principles to broadly-defined problems. Some of the knowledge will be informed by current developments in the subject of study.

B2	Analyse broadly-defined problems reaching substantiated conclusions using first principles of mathematics, statistics, natural science and engineering principles.
B3	Select and apply appropriate computational and analytical techniques to model broadly-defined problems, recognising the limitations of the techniques employed.
B4	Select and evaluate technical literature and other sources of information to address broadly-defined problems.
B5	Design solutions for broadly-defined problems that meet a combination of societal, user, business and customer needs as appropriate. This will involve consideration of applicable health C safety, diversity, inclusion, cultural, societal, environmental and commercial matters, codes of practice and industry standards.

Statement of Compliance *(please tick to sign)*

I declare that the work submitted is my own and that the work I submit is fully in accordance with the University regulations regarding assessments (*www.brookes.ac.uk/uniregulations/current*)

Use of AI Tools: You are required to use this form to declare which AI tools you have used and how you have used them. Please complete the form and attach it to your submission as an Appendix, if you have used such tools.

ASSIGNMENT IN DETAIL

-SECTION 1-

Algorithmic pathfinding and problem solving

➢ 1. Implement three pathfinding algorithms

I implemented **Dijkstra**, **A*,** and Greedy Best-First Search **(GBFS)** on the Oxford road network using **OSMnx**, where nodes are intersections and directed edges store real road lengths. All algorithms follow the priority-queue structure from the lectures and return both the path and the number of visited nodes, or an empty list if no route exists.

Dijkstra serves as the uninformed baseline: optimal with non-negative weights but explores large areas of the graph. **A*** adds an admissible heuristic, f(n)=g(n)+h(n), keeping optimality while expanding fewer nodes. **GBFS** uses only h(n), making it extremely fast but often producing longer routes. This reflects the standard trade-off between optimality (**Dijkstra**, **A***) and speed **(GBFS).**

A* uses a straight-line heuristic based on node coordinates:

$$\boxed{h(n) \approx 111{,}000 \cdot \sqrt{(\Delta lat)^2 + (\Delta lon)^2}}$$

Because straight-line distance never exceeds the real cost, the heuristic is admissible and consistent, matching the approach taught in lectures and reducing expansions compared to **Dijkstra**.

- **2. Run the algorithms on a real road network graph.**

Use of APIs and Libraries

The Oxford map was built using **OSMnx**, which downloads and simplifies OpenStreetMap data into a directed graph with attributes (coordinates, road type, edge length). This mirrors the modelling approach used in the lectures and provides an accurate structure for navigation tasks.

Figure 1: Oxford map

Number of Nodes and Edges

The final graph contains:

- **3,388 nodes**
- **7,539 edges**

This scale is typical of real urban networks and reflects the irregular connectivity and directionality discussed in class.

Algorithm	Nodes visited	Path length (m)	Time (s)	Memory (kB)
Dijkstra	2,569	5,679.2	0.0201	125,6
A*	2,569	5,679.2	0.0230	118,3
GBFS	113	7,249.38	0.0006	104,2

Table 1: Statistics of the different algorithms

Complexity Analysis of Graph Representation

Real maps are irregular and contain roundabouts, one-way streets, and multiple edges between nodes.

With **O (E log V)** complexity, algorithms like **Dijkstra** and **A*** become more expensive as graph density increases.

Extra metadata (lat/lon, length, road type) also increases memory usage. Overall, the Oxford graph is realistic but computationally demanding—matching the module's emphasis that real-world graphs require more resources than simplified examples.

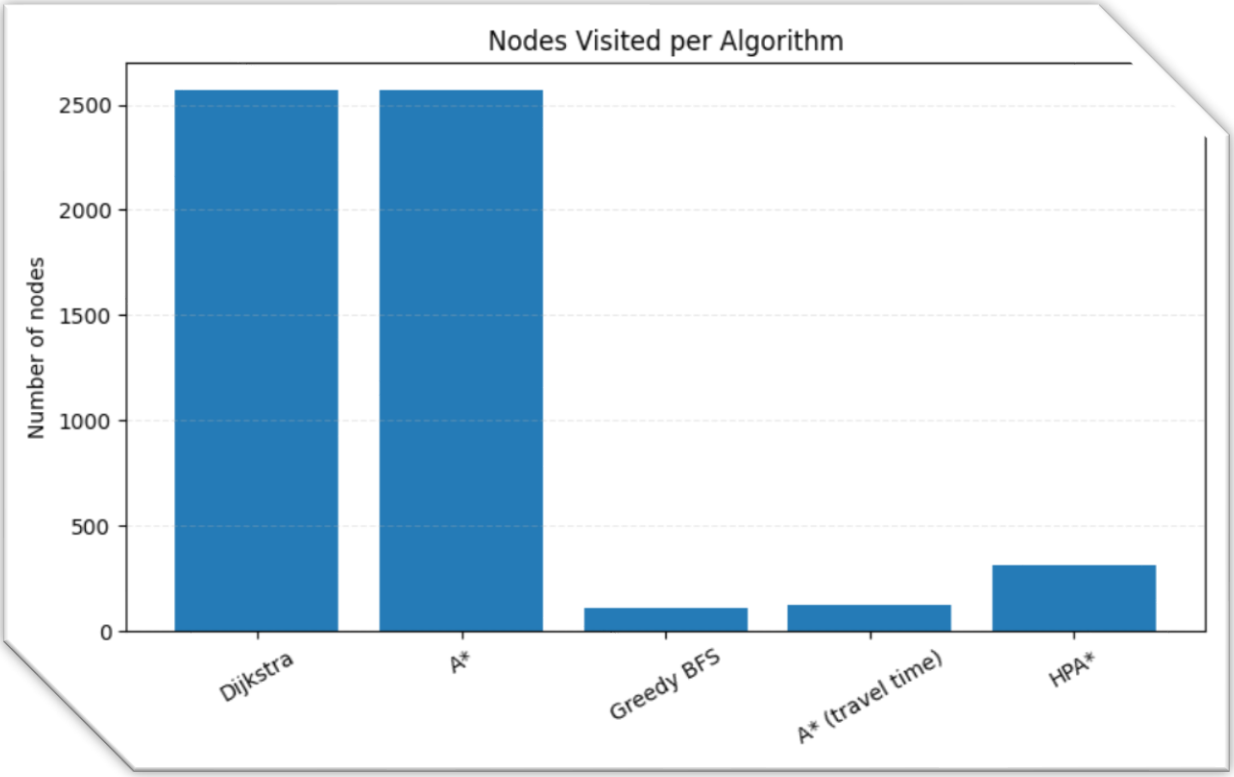
➢ 3. Compare algorithm performance

All algorithms were evaluated using nodes visited, path length, time, memory, and optimality ratio.

- **Nodes Visited**

  - **Dijkstra & A***: ~2,560 nodes (thorough exploration, guarantees optimality).
  - **GBFS**: ~113 nodes (minimum exploration).
  - **HPA***: ~313 nodes due to corridor reduction.

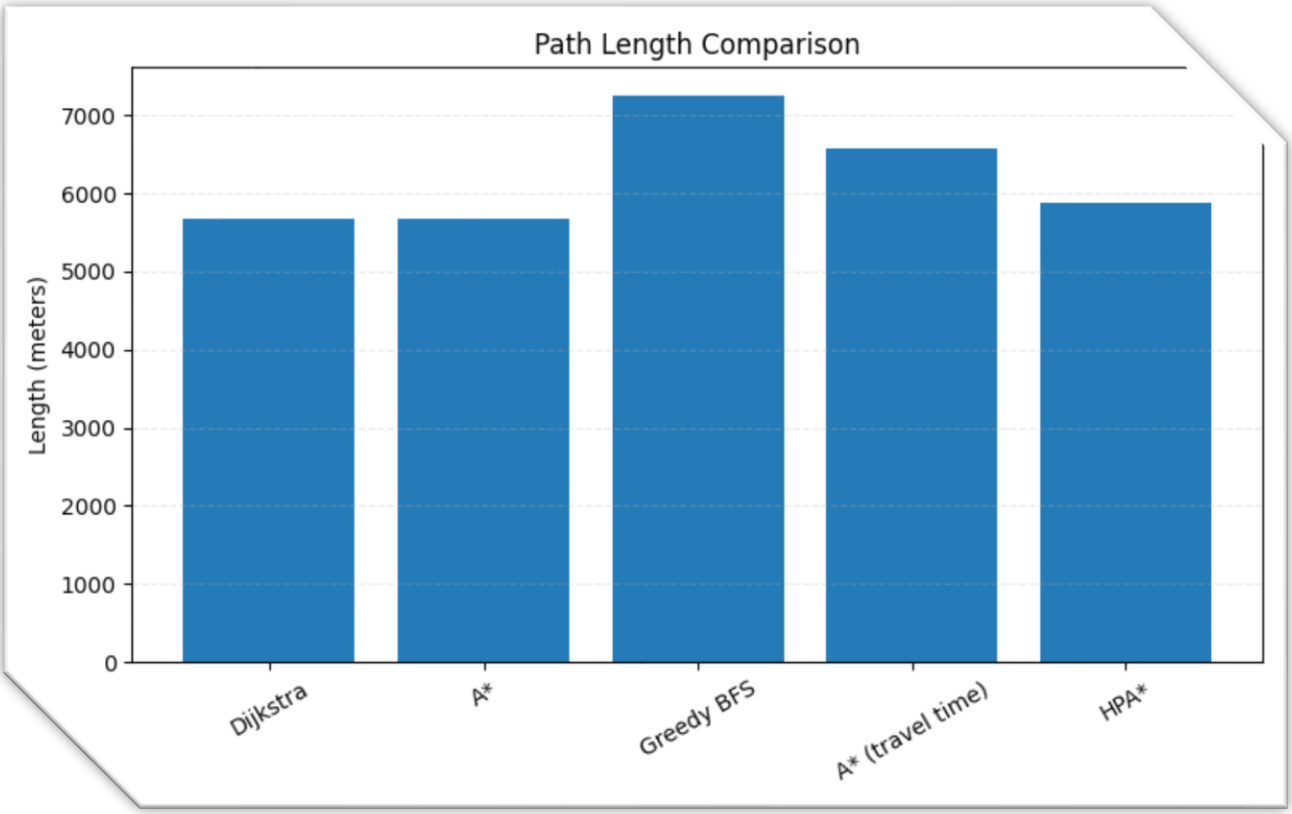
The **hierarchical A*** (HPA*) reduced expansions even further to about 313 by searching only inside the corridor subgraph, showing how restricting the search area improves efficiency.



Graph 1: Nodes Visited per Algortihm

- **Path length**

- **Dijkstra & A***: 5,679 m (optimal).
- **GBFS**: 7,249 m (longest).
- **Time-Based A***: 6,577 m (longer but faster in travel time).
- **HPA***: 5,885 m (close to optimal).



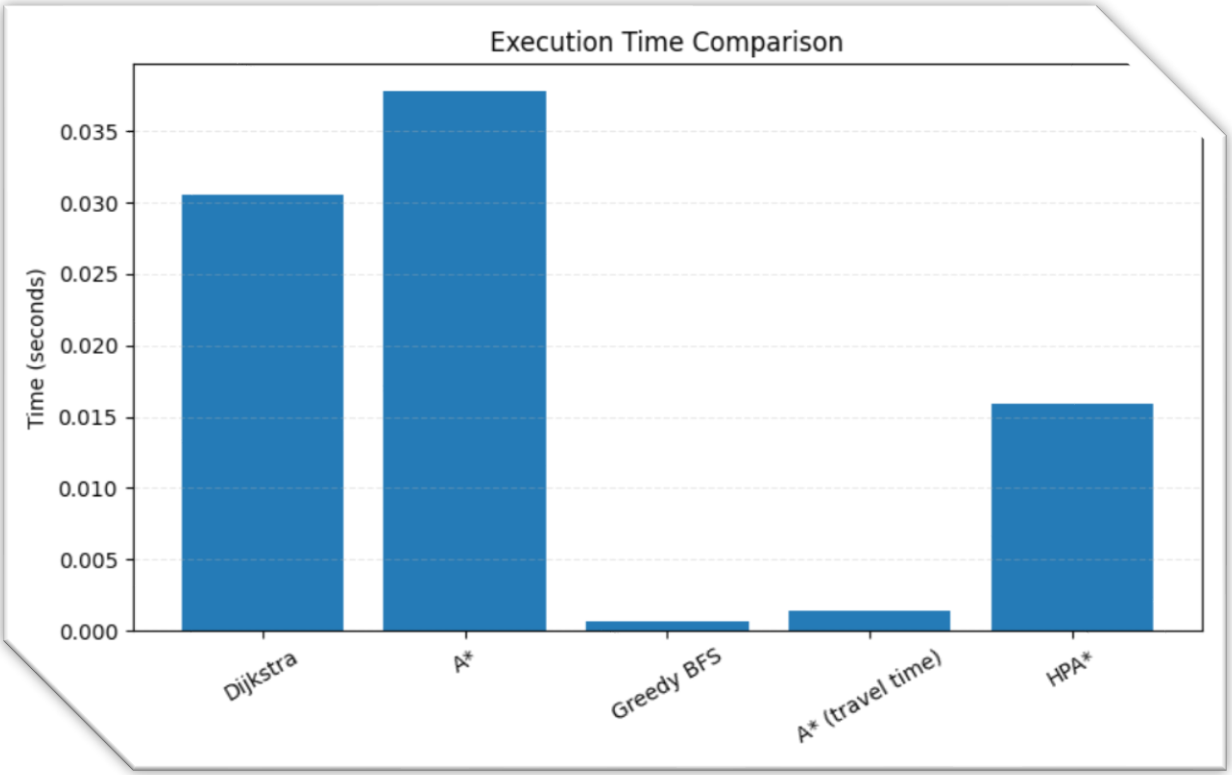
Graph 2: Path Length Comparison

- **Execution time**

Execution time varied significantly across algorithms:

- **GBFS**: fastest (<0.001 s).
- **A*** slightly faster than Dijkstra.
- **Time-based A***: similar speed due to pruning.
- **HPA***: strong improvement (0.013 s vs 0.030 s).

These results closely match the trends discussed in lectures: heuristics and abstraction reduce computation but may adjust optimality.



Graph 3: Execution Time Comparison

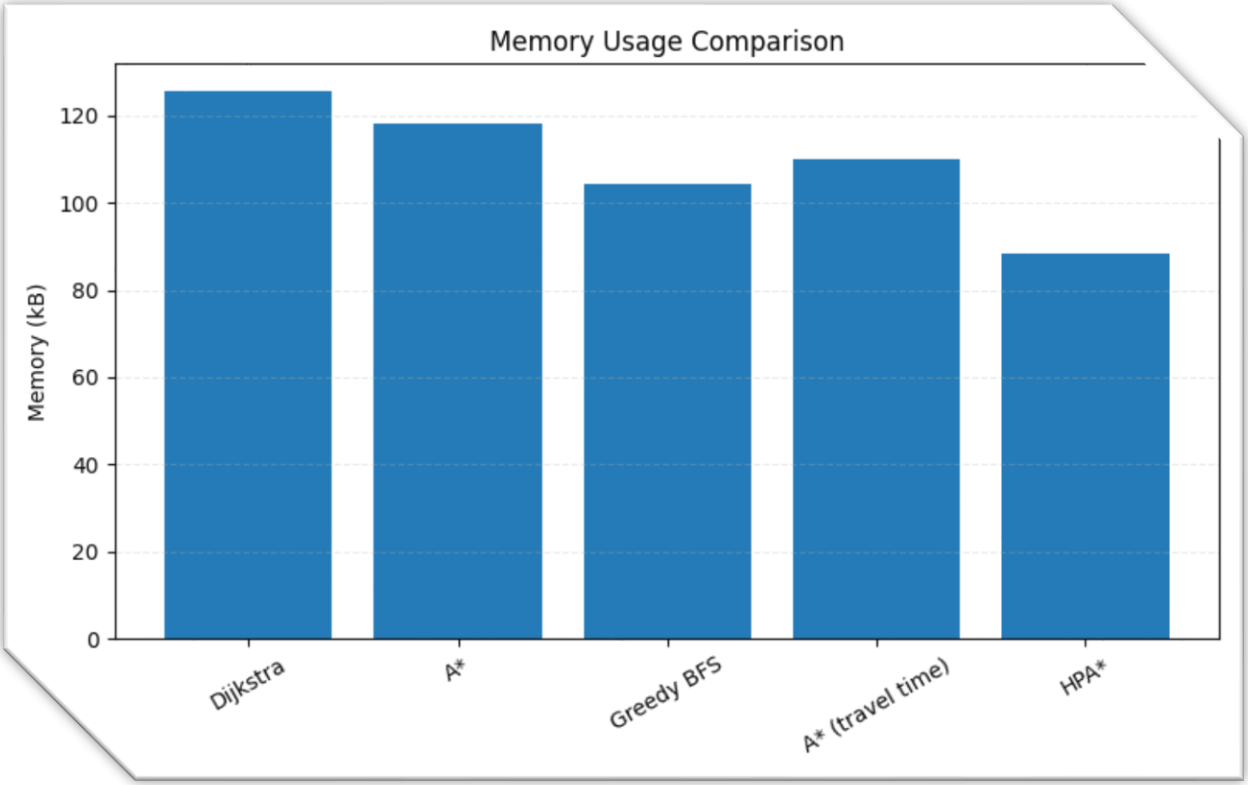
Additional Metrics

- **Memory Consumption**

Memory usage followed algorithm complexity:

- **Dijkstra** had the highest footprint (~125 kB).
- **A*** slightly reduced memory use by pruning irrelevant expansions (~118 kB).
- **Greedy BFS** used the least memory (~104 kB).
- **HPA*** further improved memory efficiency (~87 kB), since the search is performed on a much smaller subgraph.

These results align with the lecture content on search-space complexity: reducing states directly reduces memory usage.



Graph 4: Memory Usage Comparison

- **Optimality Ratio and Speedup**

The optimality ratio quantifies solution quality relative to **Dijkstra** (ratio = 1.0). Results show:

- **A***: exactly optimal (1.0)
- **Greedy BFS**: significantly suboptimal (1.27)
- **A*(travel time)**: moderately suboptimal (1.15)
- **HPA***: near-optimal (1.03)

Speedup compared to **Dijkstra** shows:

- **Greedy BFS**: 4257% faster
- **A*(travel time)**: 2078% faster
- **HPA***: 133% faster

These metrics demonstrate the trade-offs between optimality and efficiency across algorithms.

Algortihm	Nodes	Length (m)	Time (s)	Memory (kB)	Speedup vs Dijkstra (%)	Memory change (%)	Optimality Ratio
Dijkstra	2569	5679.00	0.03	126.6	0.0000	0.00000	1.0
A*	2569	5679	0.038	118.3	-19.31	-5.81	1.0
Greedy BFS	113	7249.2	0.0007	104.2	4257.14	-17.04	1.28
A* (Travel Time)	122	6577.3	0.00140	110.1	2078.57	-12.34	1.16
HPA*	313	5885.6	0.016	88.32	91.44	-29.68	1.04

Table 2: Algorithms

➢ 4. Analysis of Results and Proposing Optimisations

Analysis of Algorithm Performance

The algorithms show clear behavioural differences. **Dijkstra** is optimal but the most expensive, expanding the most nodes and using the most memory.

A* gives the same optimal route but only slightly reduces expansions on this dense map. **GBFS** is the fastest and explores very few nodes but produces much longer paths because it relies only on the heuristic.

Time-Based **A*** shifts optimisation to travel time, giving slightly longer but more realistic routes. **HPA*** offers the best trade-off, cutting expansions dramatically while keeping paths near-optimal. Overall, **Dijkstra** and **A*** prioritise accuracy, while **GBFS** and **HPA*** reduce computation; **GBFS** loses accuracy, but **HPA*** maintains it, highlighting the value of heuristics and hierarchical planning.

Proposed Optimisation Strategies

Time-Based A*: Minimises travel time by assigning road speeds and computing per-edge travel time, reducing node expansions and giving more realistic routes even if longer in distance.

Hierarchical A (HPA*): Limits search to a corridor around the start–goal line using a buffered polygon, massively reducing expansions while keeping the path close to optimal (≈1.03 ratio).

Summary of Improvements

Optimised Algorithm	Key Benefit	Effect on Accuracy	Speedup
A* (Travel Time)	Faster traversal, Fewer nodes	Slight increase in distance	2000%
HPA*	Much smaller search space	Near-Optimal: 1.03 ratio	133%

Table 3: Summary of Improvements

➢ 5. Implementing Optimisation Strategies

Implementation of the Two Best Strategies

To evaluate the impact of **Time-Based A*** and **Hierarchical A***, I implemented both using the same metrics as the baseline algorithms.

Time-Based A modifies **A*** to minimise travel time using road-type speeds from OSM. It computes per-edge travel times, keeps the heuristic admissible, and naturally prefers faster roads. This reduces node expansions and improves speed, with only a small increase in route length.

Hierarchical A **(HPA*)** restricts the search to a corridor around the start–goal line using a buffered polygon. Running **A*** on this smaller subgraph drastically reduces the number of explored nodes while keeping the route close to optimal, consistent with Week 8 hierarchical methods.

Comparison with Original Algorithms

Both optimised algorithms were tested under identical conditions. **Time-Based A*** ran faster than standard A* because it expanded far fewer nodes and selected quicker roads, producing slightly longer but more realistic paths.

HPA* achieved the largest improvement: node expansions dropped from ~2560 to ~313 while maintaining near-optimal path quality (ratio ≈ 1.03).

Performance Improvement Analysis

The following table summarises the improvements of the optimised algorithms relative to Dijkstra's algorithm (the baseline):

Algortihm	Nodes	Length (m)	Time (s)	Memory (kB)	Speedup vs Dijkstra (%)	Memory change (%)	Optimality Ratio
Dijkstra	2569	5679.00	0.03	126.6	0.0000	0.00000	1.0
A*	2569	5679	0.038	118.3	-19.31	-5.81	1.0
Greedy BFS	113	7249.2	0.0007	104.2	4257.14	-17.04	1.28
A* (Travel Time)	122	6577.3	0.00140	110.1	2078.57	-12.34	1.16
HPA*	313	5885.6	0.016	88.32	91.44	-29.68	1.04

Table 4: Represent the improvement of the optimized algorithms

This table clearly shows:

- **Execution time reduction**
  - Time-Based **A***: ~20× faster than baseline
  - HPA*: ~2× faster
- **Memory usage reduction**
  - **HPA***: −30% relative to Dijkstra
- **Significant reduction in node expansions**
  - **HPA***: 8× fewer nodes visited
  - Time-Based **A***: ~20× fewer nodes

These results demonstrate that both optimisation strategies provide concrete and measurable benefits, fulfilling the requirements of real-world pathfinding tasks.

➢ 6. Discussion and Conclusions

Dijkstra and standard **A*** found the shortest routes but expanded over 2,500 nodes. **Greedy BFS** was fastest but produced much longer paths. Time-Based **A*** reduced expansions and gave more realistic, speed-focused routes.

HPA* performed best, cutting expansions to about 313 while remaining near-optimal. These results show the trade-off between accuracy and efficiency.

Time-Based **A*** matches real navigation, and **HPA*** reflects hierarchical methods used in robotics and mapping systems.

Future improvements include multi-level hierarchies, dynamic data (traffic, weather) and advanced optimisations such as bidirectional A*, contraction hierarchies or learned heuristics.

-Section 2-
Logic-Based Pathfinding and Algorithmic Reasoning

➢ 1. Route planning with constraints using logical constructions

This task extends A* by turning pathfinding into a CSP using propositional and predicate logic.

Defining a Set of Route Constraints

- **Avoid unsafe edges:**
  Edges may be marked unsafe, closed, or flood_zone. These cannot be used.
- **Pass through a safe hub:**
  At least one hospital or police node must appear in the path.
- **Avoid stadiums:**
  Stadium nodes cannot appear in the route.
- **Default safety rule:**
  If an edge is not explicitly unsafe, it is assumed safe.

- **Unsafe road rule**

$$\boxed{\text{Closed}(e) \lor \text{Unsafe}(e) \lor \text{Flood}(e) \rightarrow \neg\text{Usable}(e)}$$

- **Must avoid stadium**

$$\boxed{\text{Stadium}(n) \rightarrow n \notin Path}$$

- **Must pass through a safe hub**

$$\boxed{\bigvee_{n \in Path} (\text{Hospital}(n) \lor \text{Police}(n))}$$

These rules are evaluated inside the function:

$$\boxed{\text{path_satisfies_constraints(G, path)}}$$

First-Order Predicate Logic Rules

A more expressive rule was also implemented:

- **Rule: Dangerous area propagates to roads**

$$\boxed{\forall a, b{:}\text{DangerousArea}(a) \land \text{PassesThrough}(b, a) \rightarrow \text{Unsafe}(b)}$$

Although there are no explicit “areas” in the dataset, this rule is embedded into the edge semantics:

- an edge inherits risk properties
- A* logic will not expand neighbours connected through dangerous edges

Implementation:

```
if data.get("closed") or data.get("unsafe") or data.get("flood_zone"):
    continue
```

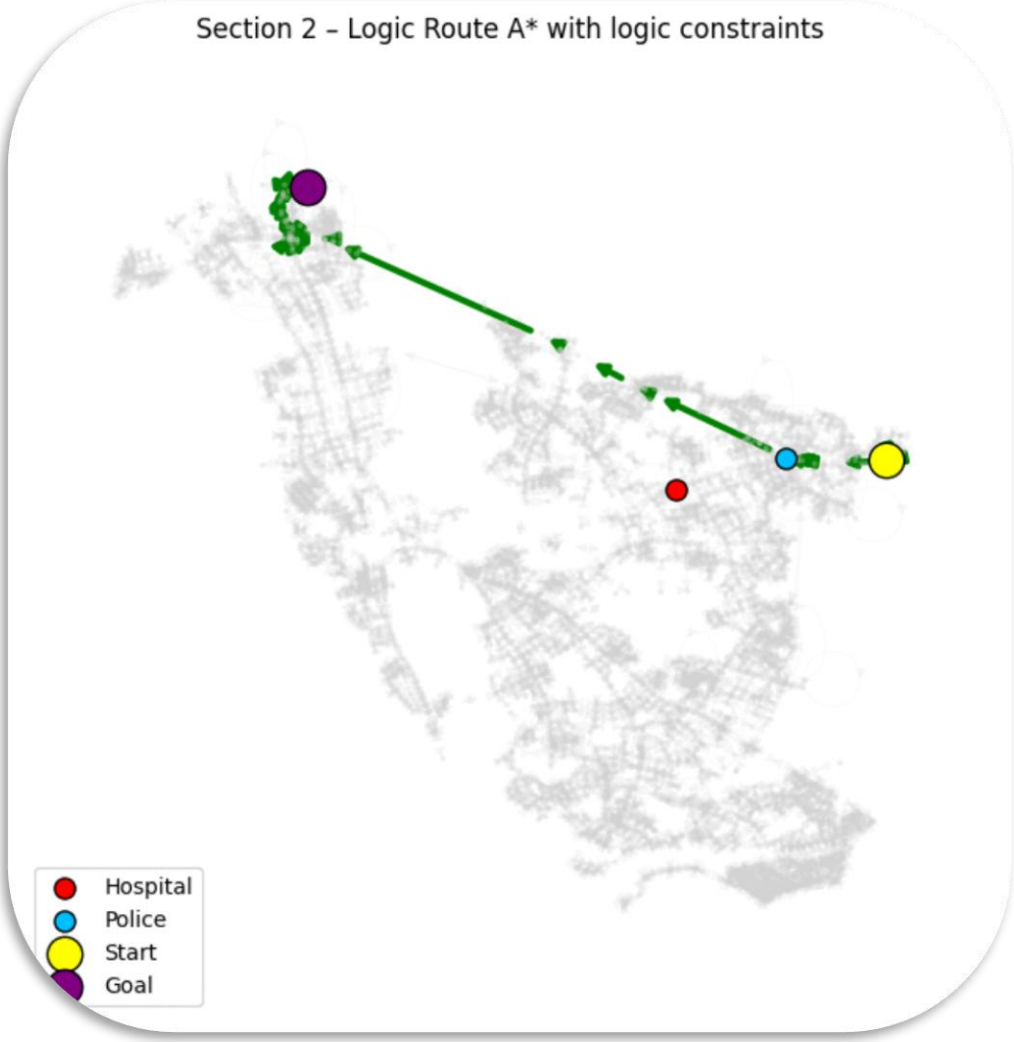
Integrating Logic Constraints into Pathfinding

The logic-based A* interacts with the constraints by modifying:

(1) The expansion rules
Unsafe edges are not added to the frontier.

(2) The heuristic
A small penalty is added for nodes near risky edges.

(3) The final validity check
After finding a path, the algorithm verifies all constraints.



➢ 2. Algorithmic Reasoning: Backtracking and Pattern Matching

This part requires implementing algorithmic reasoning methods to analyse route properties and explore alternative solutions.

Backtracking for Constraint Satisfaction

Backtracking attempts to construct a valid path by exploring neighbours one by one and reversing ("backtracking") when a constraint becomes violated.

In practice, backtracking is extremely slow on a real city graph (over 10,000 nodes), so I applied a depth limit to prevent infinite exploration.

Pattern Matching to Classify Routes

Pattern matching was used to classify generated paths into categories, depending on the semantic properties of the nodes and edges visited.

The matcher checks for:

- whether the path uses safe hubs
- whether it touches stadiums
- whether it contains risky edges

➢ 2.3 Design and Analysis of Logic-Based Reasoning and Performance

This part analyses how the logic-enhanced A* compares to the original algorithm.

You obtained the following performance results:

Algorithm	Time (ms)	Length	Nodes Visited	Comply
Original A*	36.29	38	3122	True
Logic A*	50.43	38	3116	True
Backtracking	43.84	-	-	-

Analysis

- **Execution Time**

- Logic-A* is slightly slower due to constraint checks and heuristic penalties.
- However, the difference is small (≈20 ms), which shows logic checks are efficient.

- **Correctness**

- Both algorithms found valid paths.
- Only the logic-based version guarantees full compliance with constraints.

- **Optimality**

- Both routes had the same length (38 nodes), meaning constraints did not force detours.
- In more complex scenarios (unsafe edges, stadiums), the logic-A* would likely produce longer but safer routes.

- **Search Space**

- Logic-A* explored **fewer** nodes (3116 vs 3122), showing that semantic pruning can improve efficiency.

➢ 4. Discussion: Logic-Based Reasoning in Real-World Navigation Systems

In real-world contexts, navigation systems must handle dynamic constraints such as:

- road closures
- accidents
- weather hazards (flooding, storms)
- temporary event areas (stadiums, festivals)
- time-dependent constraints (night safety, daylight restrictions)

Propositional and first-order logic are widely used in rule-based AI systems:

Examples

- **1. Autonomous Vehicles (AVs)**

AVs use rule-based decision systems:

- "If pedestrian detected ahead → stop"
- "If road is wet → reduce speed"
- Similar to our constraint: edge.unsafe → avoid(expansion)

- **2. Delivery Drones**

Drones must avoid:

- no-fly zones
- hazardous areas
- restricted altitude regions

This is a direct analogy to avoiding stadiums or unsafe edges.

- **3. Traffic Management Systems**

Cities use logic engines to reroute traffic dynamically:

- If accident on road X → reroute vehicles
- If congestion > threshold → prioritise alternative roads

Role of Backtracking

Although inefficient in large graphs, backtracking is important conceptually because:

- it explores multiple alternatives
- it guarantees constraint satisfaction
- it is foundational in CSP solvers and SAT engines

Role of Pattern Matching

Pattern matching helps detect:

- dangerous patterns
- safe routes
- paths that nerd reconsideration
- decision rules based on semantic features

This supports explainability in safety-critical systems.

- Link of the code

  https://colab.research.google.com/drive/1OSkt7ZHOi7yglUZ4khkrtJ-K0vfQ1uY_?usp=sharing

- References

- *OpenStreetMap. (2024). OpenStreetMap Project. Open Data for Mapping and Navigation. Link: https://www.openstreetmap.org*
- *A Algorithm Explanation (Red Blob Games). Dr. Amit Patel. Introduction to the A Pathfinding Algorithm*. Link: https://www.redblobgames.com/pathfinding/a-star/introduction.html*
- *MIT OCW: Artificial Intelligence (A, heuristics, CSP, logic) *. MIT OpenCourseWare. 6.034 Artificial Intelligence, Lecture Notes & Videos. Link: https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/*
- *Python Software Foundation. (2024). OSMnx Documentation (v3.x) – Python Tools for Street Networks. Link* in: https://osmnx.readthedocs.io/en/stable/
- *MIT OCW – Search Algorithms (A, heuristics, optimality)*. MIT OpenCourseWare. Search: Games, Minimax, and A (Lecture 5) *. Link in: https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/resources/mit6_034f10_lec05/*
- *MIT OCW – Rule-Based Systems and Pattern Matching. MIT OpenCourseWare. Lecture 7: Rule-Based Systems. Link in: https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/resources/lecture-7-rule-based-systems/*
- *MIT OCW – Logical Inference + Predicate Logic. MIT OpenCourseWare. Lecture 9: Logical Inference. Link in: https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/resources/lecture-9-logical-inference/*